ПРОЕКТ «МУЛЬТ ЯПОНИЯ»

Механика создания

Подробное руководство
по созданию проекта

Порядок работы, структура системы, инструкции
и проверка результатов.

Дата документа: 2026-10-25

Содержание

Нажмите на название раздела или номер страницы для перехода. В панели закладок PDF также доступны подразделы.

1. Как читать и использовать руководство . . . 3
2. Что уже решено, а что остаётся выбрать . . . 4
3. Полная схема: кто где находится и что запускает . . . 5
4. Чаты ChatGPT и автоматические работники — два разных механизма . . . 6
5. Привязка конкретного чата к роли и папке . . . 8
6. Как ChatGPT соединяется с локальным диспетчером . . . 10
7. Как диспетчер запускает и продолжает Codex . . . 11
8. Реальные границы между ролями . . . 13
9. Папки проекта и единственный основной экземпляр . . . 14
10. Память, история и восстановление контекста . . . 16
11. Карточка задания и состояния работы . . . 17
12. Инструменты MCP и обмен сообщениями . . . 19
13. Постоянная критика и возвраты на доработку . . . 20
14. Производственный цикл мультфильма . . . 21
15. Один пример от поручения до принятой сцены . . . 23
16. Инструменты генерации и учёт ресурсов . . . 24
17. Порядок сборки от пустой основы до пилота . . . 25
18. Проверки, которые подтверждают работоспособность . . . 27
19. Сбои, резервирование и продолжение работы . . . 28
20. Как Егор будет пользоваться готовой системой . . . 30
21. Когда система действительно готова . . . 31
22. Шаблоны для реализации . . . 32
23. Готовые поручения для будущей реализации в Codex . . . 35
24. Основания, источники и границы достоверности . . . 38

Подробное руководство для проекта «Мульт Япония»

Егор · 25 октября 2026 года · Версия 1.0

Цель — создать управляемую ИИ-киногруппу, в которой продюсер, режиссёр, сценарист и художник работают каждый в своей области, обмениваются результатами и замечаниями, сохраняют историю и постоянно совершенствуют материал. Егор работает через ChatGPT и Codex; самостоятельно писать программу ему не требуется.

Основа уже выбрана: координатор, программный диспетчер, отдельные исполнители, общие материалы и память проекта. Этот документ превращает основу в последовательный план сборки и проверки. Он не означает, что описанные службы уже установлены или что все предложенные детали утверждены.

Главный практический ориентир: сначала доказать на двух ролях, что система правильно адресует задания, ограничивает доступ, сохраняет результат и переживает остановку. Затем собрать всю киногруппу и провести одну пробную сцену через полный цикл с замечаниями и исправлениями.

1. Как читать и использовать руководство

Документ устроен от общего устройства к конкретной сборке. Разделы 2–8 объясняют архитектуру и главный вопрос о чатах. Разделы 9–16 описывают файлы, память, задания, обмен замечаниями и производство. Разделы 17–23 дают порядок внедрения, проверку, эксплуатацию и готовые поручения для Codex. В конце находятся шаблоны и источники.

В тексте различаются четыре статуса:

- **Принято** — прямо следует из наших обсуждений и сохранённого проекта.
- **Подтверждено документацией** — описано в прочитанной официальной документации на дату руководства; наличие в конкретной учётной записи проверяется отдельно.
- **Предлагается** — техническое решение этого руководства, которое ещё нужно реализовать и испытать.
- **Требует проверки** — нельзя обещать до наблюдаемого результата в нашей среде.

Этапы не привязаны к неделе, месяцу или дедлайну. Следующий этап начинается после выполнения критериев предыдущего. Команды и задания в приложениях предназначены для будущей реализации через Codex; при написании этого руководства они не запускались.

Краткий словарь:

Название	Что это означает в нашем проекте
ChatGPT	Интерфейс обсуждения и возможный координатор; его модель работает в инфраструктуре сервиса
Codex	Среда, через которую Егор поручает разработку; также возможный исполнитель производственных задач
Координатор	Модель, которая понимает цель, выбирает работу и разбирает результаты
Диспетчер	Обычная программа, проверяющая адресата, права, очередь, состояние и доставку

Название	Что это означает в нашем проекте
MCP	Способ дать модели набор конкретных инструментов; сам по себе не сотрудник и не планировщик
Туннель	Канал доставки вызовов к нашему MCP-серверу; не механизм распределения ролей
Рабочая сессия	Сохранённая история конкретного исполнителя; отличается от файла с итогами
Рабочая область	Файлы и инструменты, которые разрешены исполнителю
Материал	Версионируемый результат: сценарий, изображение, план сцены, видео, замечания

2. Что уже решено, а что остаётся выбрать

2.1. Принятая основа

Проект называется «Мульт Япония». Мы создаём мультфильмы и аниме в японском стиле для YouTube. Конкретное произведение, формат выпусков, длительность, визуальное решение и инструменты генерации пока не определены.

У Егора уже распределены должности и обязанности. В сохранённых материалах названы продюсер, режиссёр, сценарист и художник, но полных должностных инструкций нет. Предлагаемые далее карточки описывают техническое подключение ролей и не подменяют существующие инструкции Егора.

20 октября принята за основу схема помощника с диспетчером. На рисунке есть отдельные инструкции, рабочие области и инструменты исполнителей, общие материалы, память и история. Диспетчеру передаются исполнитель, задача и исходник. Между участниками нужны замечания и исправления.

Производство не заканчивается первой передачей результата следующему участнику. Режиссёр может вернуть сценарий, художник — обнаружить проблему в постановке, продюсер — пересмотреть объём. Любой такой возврат должен иметь адресата, основание, версию материала и сохранённое решение.

2.2. Решения, предлагаемые руководством

- Сохранить ChatGPT возможным главным интерфейсом, а программный диспетчер разместить на выбранном компьютере или сервере.
- Для автоматического исполнения использовать отдельных работников Codex с собственными сессиями и ограниченными рабочими областями.
- Для состояния первой версии рассмотреть SQLite, для материалов — обычные файлы с версиями и реестром.
- Начать с одного узла исполнения и одного экземпляра диспетчера, чтобы не вводить распределённую систему раньше необходимости.
- Отдельно проверить привязку разговоров ChatGPT к ролям через метаданные MCP.
- Постоянную критику реализовать через замечания, задания на исправление и повторную проверку; готовый Roast Room для этого не обязателен.

Это проект реализации. Python, SQLite, конкретный способ запуска Codex, набор папок и схема авторизации не объявляются окончательно утверждёнными только потому, что основа с диспетчером принята.

2.3. Что нужно определить при сборке

Где постоянно работает диспетчер; какие устройства участвуют; где лежит основной экземпляр файлов; доступен ли нужный MCP-контур в учётной записи; какие модели доступны; каким способом авторизуются исполнители; какие инструменты создают изображения и видео; каковы допустимые расходы.

Эти вопросы не мешают написать архитектуру. Они мешают честно объявить систему готовой до её проверки. Точные модели, цены и тарифные возможности в руководство намеренно не закреплены: на момент запуска их нужно проверить в выбранных учётных записях.

3. Полная схема: кто где находится и что запускает

3.1. Схема автоматической работы

```
Егор
  ↕ обсуждение и поручения
ChatGPT: координатор
  ↕ вызовы инструментов MCP
Канал доступа: Secure MCP Tunnel или согласованный HTTPS
  ↕
Наш MCP-сервер
  ↕ проверенные команды
Программный диспетчер
  ├─ реестр ролей и разрешений
  ├─ очередь, состояния, сообщения, журнал
  └─ запуск и продолжение исполнителей
       ├─ Codex: продюсер
       ├─ Codex: режиссёр
       ├─ Codex: сценарист
       └─ Codex: художник
                ↕
     Разрешённые файлы и инструменты роли
                ↕
     Материалы, версии и память проекта
```

В облаке сервиса: модели ChatGPT и Codex при обычном использовании через OpenAI. **На выбранном узле:** диспетчер, MCP-сервер, клиент туннеля, локальный процесс Codex, рабочие файлы и разрешённые программы. **У внешнего поставщика:** генерация медиа, если выбрана облачная услуга.

Установка Codex на компьютер не переносит его модель на домашнюю видеокарту. Компьютер предоставляет файлы, запускает инструменты и передаёт запросы. Локальная генерация картинок или видео — отдельный выбор, требующий конкретной модели, оборудования и настройки; она не следует автоматически из этой архитектуры.

3.2. Разделение координатора и диспетчера

Координатор получает просьбу «подготовить сцену», понимает её, предлагает разбиение и выбирает исполнителя. Диспетчер не сочиняет сцену и не оценивает художественный вкус. Он проверяет, существует ли роль, разрешено ли ей действие, доступны ли исходники, не запущено ли уже такое задание и куда записать результат.

Если координатор ошибочно просит сценариста изменить изображения художника, диспетчер не должен послушно расширить права сценариста. Он возвращает понятную ошибку или направляет запрос на создание задания художнику, если такое действие разрешено. Это предотвращает изменение чужих файлов, хотя не гарантирует, что модель всегда правильно поймёт смысл задачи.

3.3. MCP-сервер и диспетчер могут быть одной программой

На схеме это разные функции, а не требование запускать два отдельных сервера. В первой реализации один Python-процесс может принимать MCP-вызовы, проверять права и записывать задания. Работники исполняют их отдельно, чтобы длительная работа не занимала обработчик запроса.

Разделить процессы стоит, когда появится наблюдаемая причина: независимые перезапуски, несколько типов работников или требования к изоляции. Не нужно начинать с множества сетевых служб только ради красивой схемы.

3.4. Где проходит граница текущего документа

Руководство сохраняется в существующей папке проекта. Производственная служба, постоянный туннель, ключи, новые роли и платные генерации при его подготовке не создаются. Для реализации ниже даны отдельные задания с наблюдаемыми результатами.

4. Чаты ChatGPT и автоматические работники — два разных механизма

Это принципиальное различие. «Сценарист» может быть названием чата ChatGPT, ролью исполнителя Codex или записью в нашем реестре. Одинаковое имя не делает их одним автоматически синхронизированным сотрудником.

4.1. Чаты должностей внутри одного проекта ChatGPT

Егор открывает, например, чаты «Координатор», «Сценарист», «Режиссёр», «Художник», «Продюсер». Каждому задаётся инструкция роли. Через MCP чат читает разрешённые материалы, получает свою почту, сохраняет результаты и передаёт замечания другим ролям.

В этом варианте активность обычного чата начинается с действий пользователя или иных действительно доступных возможностей продукта. Наличие MCP-инструмента не доказывает, что диспетчер может по своей инициативе открыть произвольный уже существующий чат ChatGPT и заставить его работать. Такой способ автоматического управления пользовательскими чатами здесь не подтверждён.

Поэтому сообщения между ролями могут сохраняться в нашей системе, но запись в почтовый ящик сценариста не равна пробуждению его чата ChatGPT. В следующем обращении чат вызывает инструмент получения почты. Любой механизм автоматической доставки и запуска проверяется отдельно.

4.2. Автоматические работники Codex

Диспетчер действительно может запускать программно управляемые сессии Codex через поддерживаемый интерфейс. Для каждого задания он передаёт инструкцию роли, входные материалы и границы работы, а затем принимает события и результат.

Работники не обязаны иметь постоянно открытые окна. Сохранённая роль — это инструкция, разрешения, материалы, память и идентификатор сессии. Процесс можно запустить, выполнить работу и остановить; при следующем задании продолжить нужную сессию либо начать новую с сохранённой памятью.

4.3. Как совместить два способа

Предлагаемая автоматическая версия: ChatGPT служит координатором, а исполнительные задачи выполняют работники Codex. Отдельные должностные чаты ChatGPT остаются местом личного обсуждения с Егором и работы с материалами своей роли.

В первой версии ChatGPT формирует план и поручает его выполнение. Диспетчер самостоятельно исполняет зарегистрированные задания и заранее определённые переходы. Когда нужно новое смысловое решение координатора, результаты сохраняются до следующего обращения Егора в ChatGPT. Постоянно работающий диспетчер не превращает обычный чат в автоматически пробуждаемый процесс.

Для полностью автономного разбора результатов потребуется отдельный программно вызываемый координатор, например исполнитель Codex или модель через API. ChatGPT тогда останется пультом общения, а этот координатор будет выполнять рабочие циклы. Это возможное расширение, которое ещё не выбрано; его нельзя скрытно считать частью уже принятого решения.

Если «Сценарист» есть одновременно в ChatGPT и Codex, у них общий реестр материалов и решения роли, но разные истории разговоров. Для совместной работы нужен явный перенос: сохранить итог обсуждения, зафиксировать версию и передать работнику ссылку. Не следует обещать, что один автоматически прочитает всю переписку другого.

Вопрос	Чат роли в ChatGPT	Работник роли в Codex
Кто начинает работу	Егор в разговоре; другие способы требуют проверки	Диспетчер через программный интерфейс
Где история	История ChatGPT плюс явно сохранённые итоги	Сессия Codex плюс наш журнал
Как доступны файлы	Через разрешённые MCP-инструменты	Через ограниченную среду и инструменты
Как поступает замечание	Через чтение почты роли	Диспетчер передаёт его в очередном задании

Если желаемая конечная схема — именно все исполнители в обычных чатах ChatGPT с полностью автоматическим пробуждением, это остаётся отдельным условием проверки. Нельзя незаметно заменить его работниками Codex и объявить исходное требование выполненным.

5. Привязка конкретного чата к роли и папке

5.1. Что удалось подтвердить

Официальный справочник MCP-интеграций описывает поле `_meta["openai/session"]`: анонимизированный идентификатор разговора для сопоставления вызовов внутри сеанса ChatGPT. Также описаны `_meta["openai/subject"]` для пользователя и `_meta["openai/organization"]` для организации, когда она доступна. Это реальная документированная основа для исследования маршрутизации по разговору. Источник: метаданные MCP.

Но справочник не устанавливает все нужные нам гарантии: поведение при ветвлении, переносе, повторной авторизации, смене клиента, а также достаточность этого поля как самостоятельного доказательства права доступа. Поэтому вывод точный: **маркер разговора описан; надёжная привязка и защита в нашей системе должны быть реализованы и испытаны**.

5.2. Какие идентификаторы нельзя смешивать

Идентификатор	Назначение	Чего из него не следует
ID в адресе чата ChatGPT	Адрес пользовательского разговора	Что сервер получает именно его в MCP
`openai/session`	Корреляция вызовов разговора	Самодостаточная авторизация и вечная стабильность
`openai/subject`	Анонимизированное обозначение пользователя	Отдельная роль внутри его проекта
`Mcp-Session-Id`	Транспортный сеанс MCP	Тождество одному чату ChatGPT
`openai/widgetSessionId`	Экземпляр встроенного интерфейса	Идентификатор всего разговора
`threadId` Codex	Сессия исполнителя Codex	Идентификатор чата ChatGPT

Нельзя просить модель написать `chat_id` в обычных аргументах инструмента и считать это защищённым определением чата. Модель может передать неверное значение. Сервер должен отдельно обрабатывать метаданные вызова и авторизацию.

5.3. Предлагаемый порядок регистрации

1. Создать в ChatGPT нужный разговор внутри «Мульт Япония» и задать ему понятное название.
2. Подключить общий инструмент проекта. Само подключение ещё не выдаёт роли права.
3. В новом разговоре вызвать инструмент определения контекста. Сервер извлекает доступные метаданные, не возвращая ключи или токены.
4. Сервер создаёт неподтверждённую заявку на привязку: разговор, пользователь, предполагаемая роль, время и одноразовый код регистрации.
5. Егор через доверенный административный контур подтверждает, какой роли соответствует заявка. Это может быть отдельная локальная панель; её устройство предстоит реализовать.
6. В реестр записывается привязка. Только после этого чтение и запись начинают работать в пределах роли.

7. Новый или неизвестный разговор не получает чужие права по сходству имени. При отсутствии идентификатора запись запрещается, пока не выбран и не проверен иной способ привязки.

Административное назначение роли не должно быть доступно обычному исполнителю как инструмент «сделать себя продюсером». Одноразовый код служит для регистрации, а не становится постоянным секретом в переписке.

5.4. Как сервер определяет разрешение

Сначала проверяется авторизация пользователя и допустимый канал обращения. Затем сервер сопоставляет контекст разговора с зарегистрированной ролью. После этого проверяет конкретное действие, материал и разрешённую область.

Условная логика выглядит так:

```
Подтверждённый пользователь
  + принятый канал обращения
  + зарегистрированный контекст разговора
  → роль из серверного реестра
  → разрешённые действия и материалы
  → выполнение или понятный отказ
```

Поля запроса `role`, `folder`, `is_admin`, придуманные моделью, не расширяют права. Рабочий корень определяется на сервере. При несоответствии роли запрос останавливается и отражается в журнале.

Проверка OAuth выполняется отдельно от названия чата и контекстных метаданных: сервер должен проверять токен и его назначение. Официальная документация описывает такой порядок для защищённых MCP-инструментов. Источник: авторизация.

5.5. Какие проверки обязательны именно для чатов

- В двух чатах одного проекта получаются различимые контексты; сценарист не наследует привязку художника.
- В одном чате последовательные вызовы корректно сопоставляются с одной ролью.
- Переименование, закрытие и повторное открытие проверяются отдельно.
- Ветвление и новый разговор не получают права автоматически без установленного правила.
- Проверяются веб-версия и нужный мобильный клиент: одинаковое поведение нельзя предполагать.
- При повторной авторизации проверяется сохранение или контролируемое восстановление привязки.
- Попытка передать чужую роль в аргументах не меняет доступ.
- Отсутствующий, подменённый или неожиданный контекст не приводит к выдаче расширенных прав.

Нужно отдельно установить, насколько метаданным можно доверять на выбранном маршруте. Если произвольный клиент с теми же учётными данными может прислать чужое значение, одной таблицы соответствий недостаточно для строгой защиты. В таком случае нужны ограниченные полномочия соединения или другой проверяемый контур. Это не повод объявлять корреляционный маркер секретным ключом.

5.6. Что означает «доступ только этому чату»

Есть два различных результата: инструмент виден только одному чату или инструмент доступен шире, но сервер разрешает действия только зарегистрированному разговору. Руководство описывает второе как предлагаемый механизм. Документированная возможность автоматически скрывать само подключение от всех остальных чатов здесь не установлена.

Ещё одна граница: общие файлы и инструкции проекта ChatGPT доступны его чатам как общий контекст. Серверные ограничения MCP не отбирают сведения, уже помещённые в общий контекст проекта. Поэтому закрытые рабочие черновики роли не надо загружать в общие источники, если требуется ограничить их чтение. Источник: проекты и чаты.

Таким образом, сохранить несколько чатов в одном проекте и ограничивать их операции с нашими папками — обоснованная цель для прототипа. Обещать полную информационную изоляцию разговоров внутри общего проекта только за счёт MCP нельзя.

6. Как ChatGPT соединяется с локальным диспетчером

6.1. Предлагаемый канал

Вариант с Secure MCP Tunnel соответствует обсуждённой идее: на узле рядом с MCP-сервером работает `tunnel-client`. Он инициирует исходящее соединение к OpenAI, получает вызовы и передаёт их локальному серверу. Открывать входящий порт самого MCP-сервера в интернет для этой схемы не требуется. Это документированная возможность, а не подтверждение, что она уже включена в нашей учётной записи. Источник: Secure MCP Tunnel.

Для подключения проверяются три самостоятельные вещи: право пользоваться туннелем в Platform, доступ к пользовательским MCP-подключениям в ChatGPT и правильное соответствие организаций/рабочих пространств. Наличие подписки ChatGPT само по себе не доказывает наличие всех этих прав.

6.2. Последовательность настройки через Codex

1. Выбрать узел с устойчивой связью, на котором будут MCP-сервер и диспетчер.
2. Сделать локальный инструмент проверки состояния без доступа к рабочим материалам.
3. Проверить ответ этого инструмента локально.
4. Проверить доступность туннеля и необходимые разрешения в нужной учётной записи.
5. Настроить поддерживаемый клиент туннеля, указав локальный MCP-сервер.
6. Сохранить необходимые секреты в предназначенном для них хранилище; не копировать их в документ, переписку или команды отчёта.
7. Проверить работоспособность клиента и соединение с сервером.
8. Добавить пользовательское MCP-подключение в ChatGPT, выбрать туннель и требуемую авторизацию.
9. Вызвать инструмент из тестового разговора и сопоставить ответ с журналом сервера.
10. Затем переходить к регистрации ролей и доступу к файлам.

В текущей документации подключение описано через раздел «Плагины» ChatGPT, добавление пользовательского MCP-сервера и выбор туннеля. Названия и расположение элементов при реализации сверяются с реальным интерфейсом. Источник: подключение и проверка.

Если сервис сообщает об отсутствии прав, настройка останавливается с указанием отказа. Другой канал не подключается скрытно. HTTPS-вход можно рассматривать только как отдельное согласованное решение, а не способ обойти запрет.

6.3. Почему нельзя ждать готовый фильм внутри одного вызова

Транспортный запрос имеет конечное время жизни; у Secure MCP Tunnel документация указывает текущий предел одного запроса в 120 секунд. Это ограничение сервиса, а не срок выполнения нашего задания. Долгая генерация или монтаж должны продолжаться независимо от соединения.

Поэтому инструмент создания задания быстро возвращает номер и состояние. Отдельный вызов получает прогресс или результат. Закрытие ChatGPT, потеря сети или истечение времени ожидания не должны сами по себе создавать новую копию задания.

6.4. Где живёт длительная работа

Диспетчер и работники запускаются как службы выбранного узла, независимо от Telegram-бота и открытого окна чата. Остановка или обновление интерфейса не должна убивать уже принятое задание. Для существующей Linux-среды это можно организовать через службы `systemd`; конкретная конфигурация создаётся при внедрении.

Туннель даёт доступ к MCP-серверу. Он не запускает произвольную программу на любом домашнем компьютере. Если исполнение нужно на другом ПК, потребуется отдельно проверить маршрут диспетчер → этот ПК, а не только возможность ПК обращаться к серверу.

7. Как диспетчер запускает и продолжает Codex

7.1. Основной программный путь

Официальная документация описывает Python SDK `openai-codex`, который управляет локальным `codex app-server` через программный протокол. Для нашего диспетчера на Python это естественный кандидат: приложение создаёт или продолжает сессию, отправляет задание и принимает результат. Документация также описывает асинхронный клиент. Источник: Codex SDK.

SDK описан как стабильный и поставляется с закреплённой исполняющей средой Codex. Это удобнее для начальной интеграции, чем самостоятельно обслуживать все детали протокола. Прямой интерфейс `app-server` и отдельные его транспорты имеют оговорки о зрелости; их версию и поведение также необходимо проверять.

Это предлагаемый выбор реализации, а не уже установленная у нас библиотека. До разработки проверяются доступная версия, способ авторизации и поведение в выбранной среде. Точные зависимости фиксируются после рабочего испытания.

7.2. Что происходит при первом задании

1. Диспетчер находит роль в реестре и её действующую инструкцию.
2. Проверяет разрешение инициатора, версию исходников и отсутствие дубликата.
3. Формирует ограниченный пакет: задача, материалы, критерии результата, разрешённые инструменты.
4. Запускает исполнителя в среде с техническими границами доступа.

5. Через SDK или `app-server` создаёт отдельную сессию Codex.
6. Сохраняет связь «роль — задача — среда — идентификатор сессии».
7. Передаёт задание, принимает события выполнения, запросы разрешений и итог.
8. Проверяет наличие материалов и фиксирует результат отдельно от текста ответа модели.

`app-server` предоставляет операции начала и продолжения сессий, запуска хода, событий и остановки. При прямой работе с ним разработчик обязан реализовать инициализацию протокола и обработку обратных запросов; простого запуска процесса недостаточно. Источник: Codex app-server.

Точный маршрут прямого взаимодействия:

```
Запустить codex app-server
  → initialize → ответ → initialized
  → thread/start → сохранить thread.id
  → turn/start с этим threadId и заданием
  → читать события и обрабатывать запросы
  → turn/completed → проверить состояние и файлы

После перезапуска:
  thread/resume → turn/start

При остановке:
  turn/interrupt → проверить фактическое прерывание
```

При стандартном локальном транспорте программа обменивается построчными JSON-сообщениями через стандартные вход и выход процесса. Ответ на `turn/start` означает запуск, а не завершение. SDK упрощает транспорт, но наш диспетчер всё равно отвечает за очередь, права, ошибки и проверку результата.

7.3. Как продолжается работа

Для следующего задания используется конкретная сохранённая сессия нужной роли. Нельзя выбирать «последнюю сессию вообще»: она может принадлежать художнику, другому выпуску или совершенно другому проекту.

Постоянная должность не требует одной бесконечной беседы на весь проект. Предлагается сохранять сессии по сочетанию роли и рабочей темы: например, сценарист одного выпуска, художник конкретной сцены. Общие принятые решения поступают из памяти проекта. Это помогает не смешивать незавершённые версии разных выпусков.

Если история утрачена или технически непригодна для продолжения, допускается новая сессия с пакетом восстановления: инструкция, принятые решения, актуальные исходники, незакрытые замечания и точка остановки. Замена отмечается в журнале. При отказе в доступе, авторизации или запрете инструмента работа останавливается; новая сессия не используется для обхода отказа.

7.4. Запуск командой как запасной способ реализации

Для небольшого прототипа можно использовать `codex exec`: он поддерживает машинный поток событий и сохранение последнего результата. Продолжение выполняется через `codex exec resume` с конкретным идентификатором. Одноразовый режим без сохранения истории для постоянных ролей не подходит. Источник: неинтерактивный Codex.

Основной документ не предлагает параллельно строить два адаптера. Сначала выбирается один поддерживаемый способ и проходит полный набор испытаний. Прямой `app-server` имеет смысл, когда нужны его возможности и есть причина не пользоваться SDK.

7.5. Один работник не равен отдельной подписке

Несколько ролей могут использовать один выбранный тип модели, но это отдельные задания и отдельное потребление ресурса. Их количество не умножает лимиты учётной записи. Способ входа Codex и использование API определяют учёт расходов; нельзя обещать, что всё автоматически входит в текущую подписку. Источник: авторизация Codex.

Модель закрепляется в настройке роли после проверки доступности. Она не выбирается по случайному старому примеру из документации. Смена модели фиксируется в журнале задания, чтобы сравнивать результат на одинаковых исходниках.

8. Реальные границы между ролями

8.1. Три разных уровня

Инструкция объясняет, чем заниматься. **Разрешения инструмента** определяют допустимые операции. **Среда исполнения** ограничивает то, что процесс физически может прочитать или изменить. Для требуемого разделения нужны все три уровня.

Отдельная папка и указание рабочей директории не создают изоляцию сами по себе. В частности, режим Codex с записью в рабочую область не означает автоматического запрета чтения всех остальных файлов. MCP-серверы тоже имеют собственные права: ограничения команд исполнителя не заменяют проверок на сервере инструментов. Источники: разрешения Codex, параметры среды app-server.

8.2. Предлагаемая граница для автоматических исполнителей

Каждая роль работает в отдельной ограниченной среде: системной учётной записи с правильно заданными правами или контейнере с проверенными подключениями файлов. Необходимые материалы предоставляются на чтение, собственная рабочая область — на запись. Остальной личный архив и ключи других инструментов туда не подключаются.

Контейнер не даёт гарантию автоматически: нельзя подключать внутрь весь домашний каталог или административный сокет контейнерной системы и одновременно утверждать, что доступ ограничен. Реальные права подтверждаются контрольными операциями, а не названием технологии.

8.3. Предлагаемая матрица

Это техническая основа для сверки с настоящими должностными инструкциями Егора.

Область	Читать	Изменять	Как передать замечание
Общие принятые материалы	Все роли	Через управляемое принятие новой версии	Карточка изменения
Сценарий в работе	Сценарист; другим — выданная копия	Сценарист	Замечание сценаристу

Область	Читать	Изменять	Как передать замечание
Постановка и план сцены	Режиссёр; другим — выданная копия	Режиссёр	Замечание режиссёру
Визуальные материалы	Художник; другим — выданная копия	Художник	Замечание художнику
Производственные решения	Продюсер и согласованные роли	Продюсер в своих пределах	Предложение продюсеру
Реестр, права, история	По нужному представлению	Диспетчер и административный контур	Служебный запрос

Право читать чужой результат для проверки не означает права видеть все черновики роли. Диспетчер выдаёт конкретную версию для рецензии. Утверждённые материалы можно вынести в общую область, если это соответствует принятому порядку.

8.4. Как ограничить инструменты

Чату или работнику не нужен универсальный инструмент «выполнить любую команду» только ради чтения сценария. Ему нужны операции вроде получения карточки задачи, чтения разрешённого материала, сохранения новой версии и отправки замечания.

Генерация изображений и видео подключается роли через ограниченный адаптер. Право вызвать генерацию, потратить средства, опубликовать материал и удалить исходники — разные полномочия. Их нельзя неявно объединять в один ключ «полный доступ».

8.5. Как проверять защиту

Создать безвредные контрольные файлы для двух ролей и отдельный закрытый файл за пределами проекта. Из среды сценариста проверить разрешённое чтение, разрешённую запись, запрет чужой записи и запрет закрытого чтения. Повторить через MCP, прямой путь, относительный переход и символическую ссылку.

Сервер работает с идентификаторами материалов, сам определяет расположение файла и проверяет итоговый разрешённый путь. Одной проверки текста на наличие `../` недостаточно: границы должны сохраняться при ссылках, перемещениях и конкурентной замене файлов.

Результат испытания — журнал фактических успехов и отказов. Фраза модели «я не буду заходить в чужую папку» в этот журнал как доказательство не принимается.

9. Папки проекта и единственный основной экземпляр

9.1. Предлагаемая структура

Сейчас проект уже находится в `vault/projects/Мульт Япония`. Существующие обсуждения и разборы сохраняются. Следующая структура — предложение для будущего внедрения, а не сообщение о созданных каталогах. Названия новых папок согласуются перед созданием.

```
Мульт Япония/
  README.md
  Механика создания.md
```

```
Механика создания.pdf
Управление/
  Паспорт проекта.md
  Реестр ролей.md
  Права доступа.md
  Порядок работы.md
Общие материалы/
  Мир и персонажи.md
  Визуальные правила.md
  Принятые решения.md
Продюсер/
Режиссёр/
Сценарист/
Художник/
Выпуски/
  Выпуск 001/
История/
  Решения.md
  Обсуждения.md
```

Не нужно сразу создавать десятки пустых подпапок для будущих сцен. Каталог конкретного выпуска и его материалы появляются по реальной работе. Техническая база диспетчера может храниться в служебном месте отдельно от видимой структуры, но должна входить в согласованный комплект резервирования.

9.2. Где хранить главный экземпляр

При выборе одного сервера главным считается набор файлов и реестр на нём. Рабочий компьютер получает проверенную копию или работает через инструменты сервера. Если главным выбран ПК, серверная копия является резервной или явно настроенной репликой.

Нельзя иметь два независимых «главных» файла сценария и надеяться, что одинаковое имя гарантирует синхронизацию. Для каждого материала реестр хранит его идентификатор, версию, расположение и контрольную сумму. Передача между устройствами фиксируется как перенос или копирование конкретной версии.

9.3. Рабочие и принятые материалы

Роль создаёт новую версию в своей области. Проверяющий получает её на чтение. После принятия диспетчер меняет статус и указатель на действующую версию; при необходимости делает управляемую копию в общую область.

Принятая версия не переписывается незаметно. Новое содержание — новая версия, даже если исправление небольшое. Старую можно пометить заменённой, но она остаётся связанной с прежними замечаниями и произведёнными по ней кадрами.

9.4. Имена и идентификаторы

Пример человекочитаемых имён:

```
Выпуск 001 — Сценарий — версия 003.md
Сцена 002 — План кадров — версия 002.md
Кадр 002-004 — Ключевое изображение — вариант 03.png
Кадр 002-004 — Анимация — вариант 02.mp4
```

Серверные идентификаторы могут быть короткими: `EP001`, `SC002`, `SH004`, `TASK-0018`, `ASSET-0042`. Они не заменяют русские названия. Имя файла помогает человеку; идентификатор обеспечивает устойчивые ссылки при переименовании.

Слова «финал», «финал новый», «финал точно» не являются системой версий. Статус «принято» хранится в реестре вместе с конкретной версией и решением проверяющего.

10. Память, история и восстановление контекста

10.1. Четыре слоя памяти

Слой	Что содержит	Как используется
Основа проекта	Цель, принципы, роли, ограничения	При подготовке каждой значимой задачи
Принятое состояние	Действующие решения, персонажи, стиль, версии	Как актуальный источник работы
Рабочая память роли	Открытые вопросы, текущие задачи, незакрытые замечания	При продолжении её темы
Полная история	Поручения, результаты, сообщения, изменения и источники решений	Для восстановления и проверки

Краткая память не должна стирать историю. Полная история не должна автоматически загружаться целиком в каждый запрос. Диспетчер собирает достаточный пакет для конкретной работы и сохраняет ссылки на первоисточники.

10.2. Пакет запуска исполнителя

В каждое задание входят: название и инструкция роли; цель; исходные материалы с версиями; действующие решения; относящиеся к работе замечания; ожидаемый результат; область разрешённых изменений; правило приёмки.

Пример для сценариста:

```
Роль: сценарист
Задача: исправить мотивацию героя в сцене SC002
Основа: сценарий ASSET-0042, версия 3
Замечание: REVIEW-0007, от режиссёра
Сохранить: смысл финальной реплики и порядок событий
Изменить: переход между решением героя и действием
Результат: новая версия сцены + краткое объяснение
Права: собственные черновики; исходники только читать
Проверка: режиссёр по замечанию REVIEW-0007
```

Исполнитель не должен гадать, какой из нескольких сценариев актуален. Путь «возьми последний файл в папке» заменяется ссылкой на определённую принятую версию.

10.3. Что сохранять после каждого задания

- Полученное поручение и связь с инициатором.
- Версию инструкции роли и перечень входных материалов.

- Использованный способ исполнения, модель и идентификатор сессии, когда они доступны.
- Наблюдаемые события, ошибки и итоговый ответ исполнителя.
- Созданные материалы с версиями и контрольными суммами.
- Список изменений, нерешённых вопросов и предложений.
- Решение приёмки: кто, что и на каком основании принял или вернул.

Сохраняются сообщения, результаты и доступные события инструментов. Не нужно обещать запись недоступных внутренних рассуждений модели. История ChatGPT сохраняется только в том объёме, который реально получен через доступный способ экспорта или явную фиксацию.

10.4. Как новая сессия подхватывает проект

Перед началом она читает паспорт, свою инструкцию, действующие решения и точку остановки. Затем получает актуальные материалы и открытые замечания. Первая проверка — совпадает ли предлагаемое продолжение с сохранённым состоянием.

Восстановление считается успешным, если исполнитель может назвать текущий материал, его версию, незавершённую работу и основание последнего решения без повторного опроса Егора о уже сохранённых сведениях.

11. Карточка задания и состояния работы

11.1. Обязательные поля

Поле	Зачем нужно
Номер задания и проект	Однозначно найти работу
Инициатор и адресат	Понять, кто поручил и кто исполняет
Цель и ожидаемый результат	Отличить поручение от общего обсуждения
Исходники и их версии	Воспроизвести условия
Допустимые изменения	Ограничить область работы
Критерии приёмки и проверяющий	Определить, когда результат подходит
Зависимости	Не запускать работу по отсутствующему материалу
Ключ повторного запроса	Не создавать дубликат при обрыве связи
Попытки исполнения и события	Разобрать сбой без потери истории

Срок и бюджет добавляются, только если они заданы или действительно требуются выбранным платным действием. Не нужно заполнять их произвольными числами ради формы карточки.

11.2. Предлагаемые состояния

```
Создано
  → ожидает исходников / готово к запуску
  → выполняется
  → результат получен
  → на проверке
  → принято
       или
  → требуется доработка → новое связанное задание

Из активной работы возможны:
  ожидает решения / ошибка / остановлено
```

«Исполнитель закончил» не равно «результат принят». Даже корректный файл может нарушать сценарную задачу или визуальные правила. Диспетчер отдельно фиксирует техническое завершение, наличие материалов и содержательную приёмку.

11.3. Как избежать параллельной перезаписи

Два исполнителя могут читать одну версию и предлагать разные изменения. При сохранении каждый указывает базовую версию. Если она уже перестала быть актуальной, система не затирает новый результат: фиксирует конфликт или сохраняет отдельный вариант для рассмотрения.

Для первой версии достаточно одного активного автора конкретного материала и отдельной очереди замечаний. Параллельность лучше использовать на независимых материалах и независимых проверках. Количество работников определяется реальными возможностями среды и согласованными расходами.

11.4. Повтор запроса и повтор исполнения

Повторная доставка одной команды должна возвращать тот же номер задания. Внутри задания может быть несколько попыток исполнения, но каждая имеет своё состояние и причину.

Если внешняя генерация приняла заказ, а соединение оборвалось до ответа, нельзя сразу оплачивать вторую генерацию. Сначала проверяется сохранённый идентификатор операции у поставщика. Если результат неизвестен и проверить его нельзя, состояние так и обозначается; автоматическое продолжение не должно скрывать риск дубля.

11.5. Остановка

Команда остановки имеет ясную область: вся текущая работа проекта, конкретное задание или выбранная роль. Остановка проекта снимает ожидающие задания с запуска и отправляет прерывание активным исполнителям. После этого проверяется фактический результат прерывания.

Поздно пришедший результат сохраняется как материал остановленной попытки и не становится принятым автоматически. Внешняя операция может не поддерживать отмену; в этом случае система показывает, что она уже выполняется у поставщика, и не обещает мгновенного прекращения.

12. Инструменты MCP и обмен сообщениями

12.1. Минимальный набор

Названия ниже — предлагаемый интерфейс нашей программы, а не встроенные команды OpenAI.

Инструмент	Назначение	Главное ограничение
`get_context`	Показать проект, роль и доступные действия	Не раскрывает секреты
`list_tasks`	Получить доступные задания	Только разрешённая область
`get_task`	Прочитать карточку и состояние	Проверяет доступ к заданию
`read_artifact`	Получить материал по ID и версии	Не принимает произвольный путь
`submit_task`	Зарегистрировать поручение	Проверяет роль, адресата и повторы
`submit_artifact`	Сохранить новую версию результата	Проверяет автора и базовую версию
`send_message`	Передать замечание или вопрос	Только адресное сообщение, без чужой записи
`get_inbox`	Получить сообщения роли	Различает получение и обработку
`review_artifact`	Записать оценку конкретной версии	Не меняет сам материал
`stop_task`	Остановить разрешённую работу	Не выдаёт остановку за успешное завершение

Административные действия — назначение роли, изменение прав, управление ключами — находятся в отдельном доверенном контуре. Наличие инструмента в списке не отменяет серверной проверки каждого вызова.

12.2. Формат сообщения между участниками

У сообщения есть отправитель, получатель, тип, ссылка на задание, предмет и версия материала. Типы первой версии: вопрос, ответ, замечание, предложение, запрос доработки, решение.

Пример: «Режиссёр → сценарист; замечание к SC002, сценарий v3; переход к действию не мотивирован; требуется предложить исправление; сохранить финальную реплику». Это полезнее свободного «переделай сцену», потому что сохраняет основание и границы изменения.

12.3. Как участник получает сообщение

Для автоматического работника диспетчер кладёт сообщение в очередь и при следующем разрешённом запуске включает его в пакет. Если работник уже занят, новое сообщение не должно незаметно переписывать текущую цель: его можно оставить на следующий ход или явно направить как согласованную корректировку.

Чат ChatGPT получает свою почту через MCP по обращению. Нельзя утверждать, что обычный чат уже прочитал сообщение, только потому, что оно сохранено в базе.

12.4. Что считать доставкой

Диспетчер различает «сохранено», «выдано получателю» и «получен ответ». Получение сообщения инструментом не доказывает устранение замечания. Закрытие замечания связано с проверкой новой версии результата.

В пользовательском ответе достаточно краткого состояния и ссылки на результат. Подробный журнал остаётся в проекте и доступен для разбора, но не превращает каждое рабочее сообщение в технический отчёт.

13. Постоянная критика и возвраты на доработку

13.1. Основной цикл

```
Автор создаёт версию
  → проверяющие получают одну и ту же версию
  → замечания сохраняются отдельно
  → координатор разбирает противоречия
  → владельцу материала приходит задание
  → создаётся новая версия
  → проверяется выполнение замечаний
  → принятие или обоснованный следующий возврат
```

Это переносит полезную идею совета из разбора Нейта: отдельные позиции, сохранённые замечания и итог решения. При этом нашу производственную схему не следует называть установкой Roast Room или готовой архитектурой Нейта.

Если координатором остаётся обычный чат ChatGPT, новый смысловой разбор происходит при обращении Егора. Автоматически могут идти только уже порученные проверки и переходы, условия которых зарегистрированы заранее. Для самостоятельных новых решений требуется отдельно выбранный программный координатор, описанный в разделе 4.3.

13.2. У замечания должен быть предмет

Замечание включает наблюдение, влияние на результат, требуемое изменение и критерий повторной проверки. Например: «В кадре SH004 герой держит предмет другой рукой, чем в SH003; нарушается непрерывность действия; исправить положение предмета; проверить оба кадра рядом».

«Мне не нравится» можно сохранить как оценку, но оно не становится автоматически ясной технической задачей. Координатор уточняет, что именно мешает: смысл, темп, постановка, образ персонажа или качество исполнения.

13.3. Как разбирать противоречия

Если режиссёр просит сократить сцену, а сценарист считает, что исчезнет мотивация, обе позиции сохраняются. Координатор сопоставляет их с целью сцены и принятыми ограничениями, предлагает решение и определяет, кто вправе его принять.

Вопрос художественного направления, выход за согласованный объём или изменение важного принятого решения передаётся Егору. Модели не должны превращать повторение одной позиции в доказательство её правильности.

13.4. Как не получить бесконечное обсуждение

Не вводится произвольный предел «только три круга» или время остановки. Вместо этого каждый следующий круг должен отвечать на конкретное открытое замечание, новую версию или новое обстоятельство.

Если участники повторяют прежние аргументы без нового материала, спор фиксируется и передаётся принимающему решение. Если критерии выполнены и замечания закрыты, материал принимается. Новый пересмотр возможен по новой причине, которая связывается с прежним решением.

13.5. Изменение исходника после готовых кадров

Каждый производный материал хранит зависимость от исходной версии. При изменении сценария система показывает, какие планы, изображения и видео потенциально затронуты.

Она не должна автоматически переделывать весь выпуск. Сначала определяется влияние: изменились смысл, реплика, действие, внешность или только оформление текста. Затем запускаются конкретные доработки затронутых материалов. Уже оплаченные подходящие результаты сохраняются.

14. Производственный цикл мультфильма

14.1. Паспорт первого выпуска

До генерации фиксируются: цель, аудитория, основной замысел, формат, содержание, выбранные ограничения, критерии качества и ожидаемый способ показа. Длительность и количество сцен выбираются по замыслу, а не назначаются этим руководством.

В паспорте отдельно отмечаются исходные материалы, разрешённые способы их использования и открытые вопросы. Здесь не принимаются решения о публикации или коммерческом использовании; это сведения для подготовки конкретного выпуска.

14.2. Сценарная основа

Сначала формулируется, что происходит, почему это важно и какое изменение зритель должен увидеть. Затем составляются последовательность событий, сцены и реплики. Для каждой сцены полезно указать цель, конфликт или задачу, действие и итог.

Результат сценариста — текст определённой версии и список нерешённых вопросов. Проверка режиссёра отвечает, можно ли показать происходящее на экране и не приходится ли объяснять важное только пересказом.

14.3. Постановка

Режиссёр переводит сцену в план кадров: что видно, кто действует, где находится камера, что меняется внутри кадра и как кадры соединяются. Длительности указываются как рабочие параметры после выбора формата.

Сценарист проверяет сохранение смысла. Художник отмечает визуальные трудности. Продюсер оценивает реализуемость в выбранных ресурсах. Конфликты возвращаются в общий цикл замечаний.

14.4. Визуальная основа

До массовой генерации создаются устойчивые описания персонажей, окружения и визуальных правил. Для персонажа нужны различимые признаки: силуэт, пропорции, лицо, одежда, характерные предметы, допустимые состояния и отличия между ними.

Выбранный стиль описывается через наблюдаемые свойства: линия, свет, палитра, степень детализации, композиция, движение. Одного слова «аниме» недостаточно для воспроизводимого результата.

Художник сохраняет одобренные опорные изображения и связывает будущие кадры с их версиями. Возможность удерживать персонажа конкретным сервисом проверяется на серии кадров, а не по одному удачному изображению.

14.5. Раскадровка и черновая сборка

Для каждого кадра создаётся эскиз или опорное изображение. Из них собирается черновая последовательность, позволяющая оценить понятность действия, направление движения и темп до дорогой генерации всех фрагментов.

Такая сборка может быть простой последовательностью изображений с рабочими длительностями. Она нужна для проверки замысла; не требуется преждевременно доводить каждый кадр до окончательного качества.

14.6. Генерация изображений и движения

Работник получает карточку кадра, разрешённые исходники и параметры выбранного инструмента. Для каждого варианта сохраняются запрос, используемая модель сервиса, настройки, исходные изображения, номер внешней операции и результат. Если инструмент выдаёт значение случайного начального состояния, его тоже сохраняют; оно не гарантирует воспроизводимость между разными версиями модели.

Приёмка проверяет не только красоту кадра, но и соответствие действию, внешности, направлению взгляда, предметам, свету и соседним кадрам. Для видео нужно смотреть движение и весь фрагмент, а не только обложку.

14.7. Монтаж и звук

После приёмки подходящие фрагменты собираются в последовательность. Проверяются склейки, темп, непрерывность, читаемость текста и согласованность изображения со звуком, если он предусмотрен замыслом.

Способ создания звука, музыка, реплики и монтажная программа выбираются отдельно. Этот документ не назначает дополнительную профессию и не объявляет конкретный сервис обязательным. На первом пилоте часть монтажа может выполняться человеком; это честно отмечается как неавтоматизированный этап.

14.8. Проверка готового выпуска

Нужно просмотреть и прослушать весь экспортированный файл. Проверяются начало и конец, отсутствие пропавших кадров, соответствие принятой версии монтажа, технические параметры выбранного формата и наличие всех нужных материалов.

Подготовленный файл и опубликованный ролик — разные результаты. Публикация выполняется отдельным поручением с конкретной площадкой и параметрами. Готовность производства не подразумевает автоматическое размещение на YouTube.

15. Один пример от поручения до принятой сцены

Пример учебный: он показывает работу системы и не задаёт сюжет будущего мультфильма.

В примере переходы либо заранее входят в порученный план, либо подтверждаются при следующем обращении к координатору. Он не предполагает скрытого автоматического пробуждения обычных чатов ChatGPT.

15.1. Исходное поручение

Егор: «Подготовьте сцену, в которой герой обнаруживает необычный предмет и решает взять его с собой». Координатор сверяет паспорт пилота, определяет нужные материалы и регистрирует сценарную задачу. Диспетчер возвращает номер и состояние.

Если образ героя или визуальные правила ещё не приняты, система отмечает зависимость. Она может предложить временный учебный материал, но не должна незаметно объявить его окончательным образом проекта.

15.2. Первая версия сценария

Сценарист получает цель и общие решения. Создаёт версию 1, где герой видит предмет, поднимает его и уходит. Диспетчер проверяет файл, сохраняет материал и передаёт режиссёру на оценку.

Режиссёр замечает, что решение героя ничем не объясняется. Он создаёт замечание к версии 1: нужен наблюдаемый повод взять предмет. Чужой текст не переписывает.

15.3. Исправление и постановка

Сценарист создаёт версию 2 с понятным поводом. В ответе указывает, какое замечание исправлено. Режиссёр принимает изменение и составляет план кадров по версии 2. Версия 1 и замечание остаются в истории.

Художник получает план кадров и обнаруживает, что предмет в одном описании выглядит иначе. Создаёт замечание к плану. После уточнения появляется новая версия постановки, а сценарий не меняется без необходимости.

15.4. Кадры и повторная оценка

Создаются ключевые изображения. На просмотре режиссёр обнаруживает смену направления движения. Для конкретного кадра выдаётся доработка; остальные принятые изображения сохраняются.

После исправления выполняется пробная анимация и сборка. Если сервис генерации ещё не выбран, этот этап остаётся ручным или отложенным; статус системы отражает реальное состояние, а не намерение получить видео.

15.5. Что должно остаться после пилота

- Сам результат: сцена или согласованный учебный вариант.
- Сценарий, постановка и опорные изображения с действующими версиями.
- Поручения, замечания, ответы и решения приёмки.
- Связи между исходниками и производными материалами.
- Сведения о выполненных внешних операциях и фактических расходах, когда они доступны.

- Список ручных этапов и проблем, которые проявились в работе.

Успех пилота — одновременно пригодный результат и доказательство, что система может воспроизвести этот путь с сохранённой историей.

16. Инструменты генерации и учёт ресурсов

16.1. Как выбирать инструменты

Сначала определить нужные операции: текст, опорное изображение, вариация персонажа, движение кадра, монтаж. Для каждой операции проверить текущие возможности конкретного сервиса, способ входа, наличие API или другого поддерживаемого интерфейса, сохранение результата и учёт оплаты.

Название сервиса или ролик с демонстрацией не доказывают, что у нашего аккаунта есть нужная функция. Автоматизация требует испытания на нашей задаче. Конкретный поставщик в руководстве не выбирается, поэтому рекламные обещания и неподтверждённые цены не используются.

16.2. Единый адаптер внешней операции

Для каждого инструмента полезен одинаковый договор с диспетчером: принять задание, вернуть номер внешней операции, сообщить состояние, получить результат, записать возможный расход и объяснить ошибку.

В карточке хранится, поддерживает ли поставщик отмену, проверку после обрыва и защиту от повторной отправки. Если какого-то механизма нет, это известное ограничение, которое учитывается в очереди и восстановлении.

16.3. Что фиксировать по расходам

Отдельно учитываются работа моделей, генерации медиа, хранение и инфраструктура. У записи есть поставщик, операция, единицы расхода, время и источник фактического значения. Оценка до запуска и подтверждённый расход после него не смешиваются.

Итог для Егора показывается в рублях. Если поставщик считает в другой валюте, сохраняются исходная сумма, курс и дата пересчёта. Без текущей проверки нельзя ставить в паспорт произвольную стоимость сцены.

16.4. Пределы и очередь

Допустимые расходы и параллельность устанавливаются по решению Егора и ограничениям среды. Руководство не назначает ежедневный бюджет, число кругов обсуждения или срок завершения.

При отказе сервиса, исчерпании разрешённого ресурса или недоступной модели система сохраняет состояние и сообщает причину. Она не подключает иной платный сервис и не меняет способ авторизации без соответствующего поручения.

17. Порядок сборки от пустой основы до пилота

17.1. Этап А: инвентаризация без изменений

Codex читает проект, находит настоящие инструкции ролей, проверяет выбранный узел, версии инструментов, доступные способы входа и возможность MCP-подключения. Секреты не включаются в отчёт.

Результат — короткая карта фактической среды и перечень открытых вопросов. Проверено на устройстве, подтверждено документацией и пока предполагается — разные отметки. Если инструкции ролей хранятся вне доступного проекта, фиксируется пробел; их не восстанавливают выдуманным текстом.

17.2. Этап Б: конкретная схема и хранение

Выбираются основной узел, механизм запуска работников и расположение единственного главного экземпляра материалов. Согласуются названия папок. Составляются карта прав и карточки данных. После этого создаётся минимальная структура.

Результат — сохранённая схема, в которой можно проследить путь одного поручения до файла. Существующие материалы проекта сохраняются. Критерий перехода — нет неопределённости, какая программа запускает исполнителя и где появляется результат.

17.3. Этап В: две роли и доказанный доступ

Выбираются сценарист и режиссёр как удобная пара для текстового опыта. Создаются их среды и контрольные материалы. Проверяются собственная запись, выдача исходников на чтение и отказ в чужой записи.

Пока нет защиты, не стоит расширять систему на все роли и подключать реальные платные операции. Приёмка — наблюдаемые результаты проверки из тех же учётных записей и процессов, которые будут исполнять задачи.

17.4. Этап Г: диспетчер без модели

Сначала используется простой тестовый работник: получает задание и создаёт заранее определённый учебный файл. Так проверяются очередь, состояние, повторная доставка, остановка и восстановление без переменного поведения модели.

Результат — задание проходит от регистрации до проверки файла, а перезапуск не теряет запись. Только затем подключается настоящий Codex. Это позволяет отличить ошибку программы от неточного ответа модели.

17.5. Этап Д: реальный исполнитель Codex

Подключается один выбранный адаптер запуска. Исполнитель получает ограниченный пакет, создаёт простой текстовый материал, возвращает результат. Проверяются события завершения и продолжение конкретной сессии.

Результат — реальная работа с файлом, сохранённая история и доказанная остановка. Ответ «готово» без найденного файла не проходит приёмку.

17.6. Этап Е: MCP и конкретные чаты

Локально проверяются инструменты, затем канал подключения к ChatGPT. Два чата одного проекта регистрируются как разные роли. Проводятся испытания контекста, авторизации и запрета подмены.

Результат может быть одним из двух: требование подтверждено в выбранных условиях либо обнаружено конкретное ограничение. Во втором случае оставшаяся архитектура не объявляется полностью готовой; вопрос привязки сохраняется как нерешённый.

17.7. Этап Ж: обмен замечаниями и память

Сценарист создаёт учебную сцену; режиссёр возвращает замечание; сценарист делает новую версию; режиссёр принимает или уточняет. Затем сеанс исполнителя возобновляется либо заменяется новым с пакетом восстановления.

Результат — сохранённый цикл из нескольких связанных материалов без прямой записи в чужую область и без потери первоначального решения.

17.8. Этап З: восстановление после сбоев

Проверяются остановка работника, перезапуск диспетчера, обрыв туннеля, повтор команды, конфликт версий и поздний результат. Испытания идут на учебных материалах, где можно наблюдать последствия.

Результат — система после каждого сбоя показывает честное состояние и позволяет продолжить разрешённую работу. Неизвестный исход внешнего действия не превращается автоматически в повторный запуск.

17.9. Этап И: полная киногруппа

Добавляются продюсер и художник на той же проверенной основе. Их инструкции берутся у Егора; инструменты подключаются по необходимости. Проверки доступа повторяются для новых ролей и типов материалов.

Результат — роли работают с общей базой принятых решений, но не подменяют друг друга. Любая новая операция имеет понятный путь приёмки и журнал.

17.10. Этап К: производственный пилот

Выбирается одна сцена с достаточной сложностью, чтобы проверить реальный обмен между ролями. Фиксируются содержание, инструменты, критерии и допустимые расходы. Выполняется полный согласованный цикл.

Пилот не обязан сразу автоматизировать каждый шаг. Он должен показать, какие этапы уже работают, где требуется человек и что мешает следующему результату. Масштабирование начинается после разбора этих наблюдений.

18. Проверки, которые подтверждают работоспособность

18.1. Как оформлять испытание

Каждая проверка записывается одинаково: условия, действие, ожидаемый результат, фактический результат и свидетельство. Свидетельством могут быть сохранённый файл, запись состояния, событие исполнителя или точный ответ инструмента без секретов.

Ниже — критерии будущей приёмки. Это не список тестов, уже выполненных при написании руководства.

Проверка	Действие	Условие успеха
Адресация	Послать задачу сценаристу	Запущена его среда и нужная сессия
Ошибочная роль	Передать неизвестного адресата	Отказ без запуска случайного работника
Два чата	Вызвать инструмент из разных чатов проекта	Контексты не смешаны
Неизвестный чат	Обратиться без регистрации	Нет автоматического доступа к материалам
Подмена аргумента	Указать чужую роль в запросе	Права остались у настоящего контекста
Собственная запись	Создать черновик своей роли	Файл сохранён и зарегистрирован
Чужая запись	Попытаться изменить файл другой роли	Операция отклонена технически
Закрытое чтение	Прочитать контрольный закрытый файл	Содержимое не возвращено
Обход пути	Использовать ссылку или выход из корня	Граница доступа сохраняется
Повтор команды	Повторить тот же ключ запроса	Возвращён прежний номер задания
Конфликт версий	Сохранить поверх изменившейся основы	Нет тихой потери новой версии
Неуспешный исполнитель	Завершить работу ошибкой	Состояние «ошибка», сохранена причина
Частичный результат	Остановить после записи части файлов	Частичное явно отделено от принятого
Пропавший файл	Вернуть «готово» без результата	Проверка не принимает задание
Потеря связи	Оборвать доступ к диспетчеру	Нет дубликата принятой задачи
Перезапуск	Перезапустить диспетчер во время работы	Состояние сверено с реальным работником

Проверка	Действие	Условие успеха
Остановка	Остановить активное и ожидающее	Новые запуски прекращены; активное проверено
Поздний ответ	Доставить итог после отмены	Он не принят автоматически
Обновлённый исходник	Изменить версию сценария	Видны затронутые производные материалы
Новая сессия	Продолжить по памяти проекта	Сохранены решения и точка остановки
Принятие	Проверить конкретную версию	Статус связан с ней и проверяющим
Резервная копия	Восстановить в отдельном месте	Материалы и связи читаются согласованно

18.2. Отдельная проверка качества результата

Технически успешный запуск не доказывает качество мультфильма. Для пилота нужен свой перечень: понятность истории, узнаваемость персонажа, непрерывность действия, соответствие визуальным правилам, отсутствие заметных дефектов и пригодность итоговой сборки.

Каждый критерий связывается с конкретным материалом. Если замечание относится к движению, проверяется весь фрагмент; если к диалогу — звучащая или текстовая версия, используемая в выпуске. Оценка неподходящего представления не закрывает замечание.

18.3. Что сообщать Егору после этапа

Кратко: что теперь работает, чем подтверждено, что остаётся непроверенным и какое решение требуется, если без него нельзя продолжить. Полный технический протокол сохраняется в проекте.

Число успешных тестов само по себе не важнее их смысла. Один реальный отказ чужой записи полезнее десятка проверок, которые только сравнивают текст инструкции роли.

19. Сбои, резервирование и продолжение работы

19.1. Перезапуск диспетчера

После старта диспетчер читает незавершённые записи и сверяет их с живыми процессами и внешними операциями. Запись «выполняется» из базы не доказывает, что процесс ещё жив. Отсутствие соединения не доказывает, что внешняя операция не выполнена.

Для текущих задач полезны отметка последней активности и идентификатор владельца исполнения. При восстановлении новый диспетчер не запускает второго работника, пока не разобрал состояние первого. Это особенно важно при автоматическом перезапуске службы.

19.2. Согласованная запись файла и реестра

Файл сначала создаётся как подготовленный результат, проверяется и получает контрольную сумму. Затем становится зарегистрированной версией. Если запись прервана между этапами, восстановление должно распознать незавершённый материал.

Нельзя ограничиться записью «успешно» в базе до сохранения самого файла. Обратная ситуация тоже требует обработки: файл есть, а регистрация не завершена. Для первой версии нужен простой механизм сверки таких случаев.

19.3. Резервная копия

В комплект входят материалы, реестр состояний, роли, принятые решения и история. Настройки восстановления сохраняются без раскрытия секретов. Рабочую SQLite-базу резервируют согласованным способом, а не произвольным копированием одного файла в момент записи.

Частота резервирования выбирается отдельно по характеру работы и допустимой потере изменений. Руководство не вводит расписание. Важные контрольные точки — перед изменением структуры и после принятия значимого результата.

Проверка резервной копии — восстановление в отдельную область и чтение нескольких связанных цепочек: задание, исходник, результат, замечание, решение. Сам факт существования архивного файла не доказывает восстановимость.

19.4. Что не удалять автоматически

Старые версии, исходные изображения, видео и звук не удаляются ради «чистоты» по инициативе исполнителей. Сначала определяется, что сохраняется и что перестанет быть доступно для перепроверки, затем выполняется отдельное поручение.

Для исходных видео и кадров действует принятый порядок проекта помощника: до удаления объяснить по каждому ролику, что останется и что пропадёт, включая необходимость повторного скачивания и разбора при будущей проверке. Сохранённое описание не заменяет оригинальный материал.

19.5. Отказ доступа и обычный сбой — разные случаи

Кратковременный сетевой сбой можно повторить с тем же ключом запроса, чтобы не создать дубль. Ответ «доступ запрещён», отзыв полномочий или отказ сервиса фиксируется как причина остановки. Нельзя пытаться выполнить то же действие другим ключом или каналом без отдельного поручения.

19.6. Таблица типичных проблем

Наблюдение	Что проверить первым	Какое состояние сохранить
ChatGPT не видит инструмент	Подключение и доступность MCP-сервера	Соединение не проверено
Туннель виден, вызов не проходит	Клиент туннеля, сервер, авторизацию	Ошибка конкретного вызова
Задание долго без результата	Реальный процесс и последнее событие	Выполняется или исход неизвестен
Работник ответил без файла	Ожидаемые материалы и журнал записи	Результат не прошёл проверку
Пришло два результата	Номер задания, попытки, ключ повтора	Конфликт или поздняя попытка
Роль видит лишние файлы	Права среды и MCP отдельно	Нарушение границы доступа
Новый чат получил чужую роль	Регистрацию и доверие метаданным	Привязка не подтверждена
Генерация оплачена, файла нет	Номер операции у поставщика	Ожидается сверка внешнего результата

20. Как Егор будет пользоваться готовой системой

20.1. Начало работы

Егор открывает координатора и формулирует цель обычным языком: «Покажи, на чём остановились», «Подготовим сцену», «Разбери замечания режиссёра». Координатор читает реальное состояние через инструмент, а не восстанавливает его по смутному воспоминанию.

В ответе показываются текущий результат, незавершённые вопросы и необходимые решения. Если можно продолжить уже порученную работу, система продолжает в установленных пределах.

20.2. Разговор с отдельной должностью

В чате сценариста обсуждается сценарий, в чате художника — визуальное решение. Каждый чат может получить свои материалы и почту через MCP. Действующий контекст роли виден по запросу, чтобы ошибочная адресация была заметна.

Если поручение не относится к роли, система объясняет это и направляет запрос через предусмотренный порядок. Она не должна делать вид, будто художник превратился в администратора всей системы только из-за текста сообщения.

20.3. Утверждение результата

Егор видит конкретный материал и версию. Его решение привязывается именно к показанному варианту. Если после просмотра появилась другая версия, система не переносит согласие автоматически.

Принятие может быть частичным: например, образ персонажа утверждён, а фон требует доработки. Тогда создаются отдельные записи, чтобы не переделывать принятую часть без основания.

20.4. Возвращение через длительный перерыв

Система поднимает паспорт, последние принятые решения, материалы и точку остановки. Краткий ответ объясняет, что уже есть и какой вопрос открыт. Старое незавершённое предложение не превращается в срочную задачу только из-за прошедшего времени.

20.5. Уведомления о завершении

Для фоновых заданий заранее определяется доступный и разрешённый способ показать итог. Нельзя обещать, что любой MCP-сервер сам допишет сообщение в обычный чат ChatGPT после его закрытия.

Минимальный вариант — запрос состояния из того же чата. Автоматическое уведомление в Telegram или другом канале добавляется только как отдельно порученная возможность с подтверждением доставки и записью в историю.

21. Когда система действительно готова

21.1. Готовность архитектуры

Известны места запуска, маршрут задания и результата, границы прав, расположение файлов и способ продолжения истории. Все неизвестные вопросы записаны. Это готовый проект решения, но ещё не рабочая система.

21.2. Готовность технического контура

Две роли выполняют настоящую задачу, сохраняют результат, обмениваются замечанием и продолжают работу после перезапуска. Чужая запись и закрытое чтение отклоняются. Повторы, остановка и конфликт версий обработаны.

Для заявленной работы через ChatGPT также должен успешно пройти реальный вызов из нужного чата через выбранный канал. Локальный тест MCP без ChatGPT недостаточен для такой приёмки.

21.3. Готовность киногруппы

Все необходимые роли подключены к собственным инструкциям и инструментам. Они работают с общими принятыми решениями, оставляют адресные замечания и создают версионированные материалы. Видно, кто принимает каждую разновидность результата.

21.4. Готовность производства

Один согласованный пилот прошёл полный нужный путь и просмотрен. Известны реальные ручные этапы, ограничения качества, расходы и восстановление после сбоев. Только тогда можно обоснованно увеличивать объём.

21.5. Что не является доказательством

Нарисованная схема, созданные папки, настроенные названия ролей, успешный ответ одного инструмента и сообщение «я всё сделал» не доказывают готовность всей системы. Каждый подтверждает лишь свою часть.

На дату этого руководства сохранён проект и разработана механика создания. Производственный контур по документу ещё не развёрнут; описанные испытания и пилот предстоят.

22. Шаблоны для реализации

Все шаблоны ниже предлагаются как заготовки. Квадратные скобки обозначают поля для заполнения, а не уже выбранные значения.

22.1. Паспорт проекта

```
Название: Мульт Япония
Цель: [какой результат создаём]
Аудитория и способ показа: [принятое решение]
Формат: [выбранный формат]
Художественные принципы: [ссылки на решения]
Роли: [инструкции Егора]
Кто принимает результаты: [правило]
Основное хранилище: [выбранное место]
Разрешённые инструменты: [проверенный перечень]
Ограничения: [заданные условия]
Открытые вопросы: [нерешённое]
Точка остановки: [что завершено и что следующее]
```

22.2. Техническая карточка роли

```
Название: [сценарист / режиссёр / художник / продюсер]
Назначение: [из настоящей инструкции]
Инструкция: [файл и версия]
Способ работы: [чат / автоматический работник / оба]
Разрешено читать: [области и виды материалов]
Разрешено создавать: [собственные результаты]
Разрешено изменять: [конкретные объекты]
Инструменты: [разрешённый набор]
Полномочия принятия: [границы]
Кому отправлять вопросы: [адресаты]
Связь с сессиями: [реестр, без секретов]
Рабочая память: [расположение]
```

Правило исполнения для включения в роль: «Работай по указанной версии исходника. Чужие материалы не переписывай; замечания передавай владельцу. Новое решение отличай от предложения. В результате укажи созданные материалы, изменения и нерешённые вопросы. Если прав или исходников недостаточно, сохрани причину».

Это полезная инструкция поведения, но технические права реализуются отдельно.

22.3. Карточка задания

```
Номер: [TASK-...]
Проект и выпуск: [идентификаторы]
Инициатор: [подтверждённый контекст]
Исполнитель: [роль]
Цель: [одно конкретное изменение или результат]
Исходники: [ID + версия каждого материала]
Что сохранить: [неизменяемые условия]
Что допускается изменить: [границы]
Ожидаемый результат: [файлы и пояснение]
Критерии приёмки: [наблюдаемые признаки]
Проверяющий: [роль или Егор]
Зависимости: [предыдущая работа]
Повторный запрос: [служебный ключ]
```

22.4. Карточка результата

```
Задание: [TASK-...]
Попытка исполнения: [идентификатор]
Состояние: [фактическое]
Исходная версия: [ссылка]
Созданные материалы: [ID, версия, путь, сумма]
Что изменилось: [краткий список]
Какие замечания учтены: [номера]
Что не выполнено: [причины]
Новые вопросы: [если появились]
Требуется проверка: [что и кем]
```

Успешный ответ работника сначала проверяется программно: соответствует ли структура договору, существуют ли файлы, допустимы ли пути и версии. После этого проводится содержательная оценка.

22.5. Карточка замечания

```
Номер: [REVIEW-...]
Автор и адресат: [роли]
Материал: [ID и точная версия]
Наблюдение: [что обнаружено]
Влияние: [почему это мешает]
Предлагаемое изменение: [что требуется]
Сохраняемые условия: [что нельзя потерять]
Как проверить исправление: [критерий]
Ответ автора материала: [позиция]
Решение: [принято / отклонено / нужен разбор]
Закрыто новой версией: [ссылка после проверки]
```

22.6. Карточка кадра

```
Выпуск, сцена, кадр: [EP / SC / SH]
Основа: [сценарий и постановка с версиями]
Смысл кадра: [что зритель должен понять]
Действие: [начало → изменение → итог]
Персонажи и предметы: [опорные материалы]
Окружение и свет: [описание]
Камера и композиция: [решение]
Движение: [что и как меняется]
Связь с соседними кадрами: [непрерывность]
Рабочая длительность: [после выбора формата]
Инструмент и параметры: [проверенные значения]
Критерии приёмки: [наблюдаемые признаки]
Варианты и принятый результат: [ссылки]
```

22.7. Журнал принятого решения

```
Номер и дата: [идентификатор]
Вопрос: [что решали]
Контекст: [задача и материалы]
Рассмотренные варианты: [кратко]
Решение: [что принято]
Основание: [почему]
Кто принял: [полномочный участник]
Затронутые материалы: [список]
Что заменено: [предыдущее решение]
Что осталось открытым: [отдельно]
```

22.8. Минимальная схема данных диспетчера

Для разработчика предлагаются следующие сущности. Имена таблиц здесь служат пояснением; готовая база не создана.

Сущность	Что хранит
Роли	Инструкции, разрешения, настройки среды
Привязки разговоров	Подтверждённый пользователь, контекст, роль, история изменений
Сессии исполнителей	Роль, тема, способ запуска, конкретный ID, состояние
Задания	Цель, исходники, адресат, состояние, ключ повторов
Попытки	Реальный запуск, события, ошибки, завершение
Материалы и версии	Владельца, путь, контрольную сумму, основу, статус
Зависимости	Какие версии использованы для результата
Сообщения	Отправителя, адресата, предмет, доставку и ответ
Замечания	Предмет проверки, требуемое изменение и решение
Решения	Принятую позицию и основание

Сущность	Что хранит
Внешние операции	Поставщика, номер, статус и фактический расход

Секреты авторизации не размещаются в этих открытых карточках. Файлы и записи базы связываются идентификаторами; восстановление проверяет, что ссылки ведут на существующие версии.

23. Готовые поручения для будущей реализации в Codex

Это задания для будущего внедрения: нынешнее поручение ограничено руководством. Передавай их Codex по одному после приёмки предыдущего этапа. Программирование, настройку и проверки выполняет Codex.

Python, SQLite, MCP и локальные исполнители — предлагаемое развитие основы с диспетчером. Их применимость проверяется при реализации. Названия новых папок также согласуются при реализации. Сроки, бюджеты и количественные ограничения моделей здесь не устанавливаются.

1. Проверить исходную среду и доступные способы подключения

Поручение Codex:

«Цель: установить, на чём мы действительно можем собрать “Мульт Япония”. Прочитай проект и имеющиеся инструкции ролей. Проверь ОС, Codex, вход, сохранение сессии и запуск исполнителя. По официальной документации проверь нужный SDK, `codex app-server` и подключение MCP к ChatGPT.

Рамки: пока ничего не устанавливай и не подключай.

Результат: возможности, ограничения и предлагаемый основной путь запуска. Приёмка: возможности различены как испытанные и описанные в документации. Подписка ChatGPT не приравнена к оплате API, местный процесс — к местным вычислениям модели».

2. Описать конкретную архитектуру до её сборки

Поручение Codex:

«Цель: превратить выбранную основу в проверяемую схему. Опиши путь: ChatGPT → MCP → Python-диспетчер → отдельный исполнитель Codex → результат. Покажи размещение компонентов, состояние в SQLite и материалы в файлах. Учти инвентаризацию.

Рамки: не предполагай, что MCP автоматически открывает и ведёт существующие чаты ChatGPT. Пока координатором служит обычный ChatGPT, исполняй зарегистрированный план, а новые смысловые решения сохраняй до обращения Егора; автономный координатор является отдельным расширением. Различай идентификаторы и истории чата, роли, задания, сеанса Codex и файла. Предложи папки: “Управление”, “Общие материалы”, “Продюсер”, “Режиссёр”, “Сценарист”, “Художник”, “Выпуски”, “История”.

Результат: схема запуска, карта данных и прав, перечень этапов. Приёмка: понятно, кто решает и кто выполняет. Схема подготовлена для согласования; новые каталоги пока не созданы».

3. Подготовить согласованное хранение материалов

Поручение Codex:

«Цель: создать основу хранения по согласованной схеме и названиям. Подготовь структуру, реестр ролей и правила именования сцен и версий. Раздели рабочие и утверждённые материалы, историю решений. Сохрани существующие файлы и ссылки.

Рамки: используй инструкции Егора; пробелы отмечай, дополнения обозначай черновиками. Не усложняй иерархию заранее. Исключи бесконтрольную перезапись общих материалов.

Результат: рабочая структура и понятная карта “кто создаёт, кто читает, кто утверждает”. Приёмка: по пробному файлу можно определить автора, версию, статус, исходное задание и место хранения; существующие материалы доступны».

4. Обеспечить настоящую изоляцию исполнителей

Поручение Codex:

«Цель: ограничить доступ ролей средствами ОС или контейнеров. Выбери механизм для нашей среды. Каждой роли дай запись в собственную область и чтение разрешённых исходников. Доступ к чужим результатам — по карте прав; секреты диспетчера не передавай.

Рамки: инструкция и рабочая папка не являются изоляцией. Проверь терминал, инструменты, символические ссылки и вложенные каталоги. Обмен результатами выполняй через контролируемые операции.

Результат: настройки и проверка из-под реальных учётных записей исполнителей. Приёмка: сценарист пишет свой файл; запись в файл художника и чтение секрета отклоняются средой с записью причины».

5. Создать диспетчер и журнал заданий

Поручение Codex:

«Цель: реализовать Python-диспетчер с состоянием в SQLite. В карточке храни идентификатор, роль, цель, версии исходников, ожидаемый результат, состояние и связь с предыдущей работой. Определи состояния от ожидания до принятия, доработки, остановки и ошибки.

Рамки: диспетчер проверяет и направляет операции; творческие решения остаются моделям и человеку. Повтор запроса не создаёт второй запуск. Журналируй переходы, отказы и конфликты.

Результат: работающая очередь на тестовом исполнителе и доступный просмотр состояния. Приёмка: повтор запроса не дублирует работу; после перезапуска видны реальное состояние и история, без придуманного “завершено”».

6. Подключить настоящего исполнителя Codex

Поручение Codex:

«Цель: заменить тестового исполнителя отдельным процессом Codex через официальный интерфейс, выбранный и проверенный на первом этапе. Реализуй запуск, события, завершение и восстановление сессии. Используй ограниченную среду и отдельный контекст роли. Сохраняй связь роли, задания и сеанса.

Рамки: возобновление нужно испытать. При утрате истории создай новую сессию с контекстом из файлов и отметь замену. При отказе доступа остановись; новую сессию не используй для обхода отказа. Остановка прекращает выполняемую и ожидающую работу в указанной области.

Результат: реальный исполнитель выполняет безопасное файловое поручение и возвращает проверяемый результат. Приёмка: сохранены исходное задание, идентификатор сессии, выходной файл и итог; остановка проверена на выполняемой и ожидающей работе».

7. Подготовить память и рабочий пакет роли

Поручение Codex:

«Цель: обеспечить продолжение работы независимо от длины предыдущего разговора. Собирай пакет запуска: инструкция роли, задание, общие решения, исходники, замечания и версии. Полную историю храни отдельно от краткого актуального состояния.

Рамки: различай решения, предложения, отклонённые варианты и вопросы. Не удаляй историю ради сводки; общие правила меняй через процедуру принятия. Не включай личные материалы других проектов.

Результат: воспроизводимая сборка контекста и запись результата каждой роли. Приёмка: новый сеанс продолжает пробную задачу с действующими решениями, не подменяя их отклонёнными; источники решений доступны по ссылкам».

8. Сделать MCP-доступ с проверяемыми полномочиями

Поручение Codex:

«Цель: предоставить ChatGPT ограниченные операции диспетчера: посмотреть состояние, получить разрешённый материал, передать задание или замечание, запросить доработку. Проверяй полномочия роли на сервере при каждом вызове.

Рамки: имя роли в запросе не даёт прав. Если доступен `_meta["openai/session"]`, это признак связи обращений, не удостоверение личности и не доказанный ID чата. Авторизация требует отдельного подтверждённого механизма. Не записывай секреты в историю.

Результат: MCP с описанными операциями и журналом разрешённых и отклонённых вызовов. Приёмка: разрешённая операция проходит, чужая запись и подмена роли отклоняются сервером; отсутствие метаданных не расширяет права и не приводит к угадыванию роли».

9. Проверить связь с конкретными чатами ChatGPT

Поручение Codex:

«Цель: проверить именно работу нескольких чатов внутри одного проекта ChatGPT. Настрой согласованный MCP-канал по официальной документации, проверь реальный вызов из ChatGPT и проведи опыт с двумя чатами разных ролей. При отказе прав остановись, не заменяй канал обходным путём. Проверь метаданные при повторных обращениях и смене чата. Истории ChatGPT и Codex храни раздельно.

Рамки: чаты ведёт человек. Не подменяй проверку разделением проектов. Регистрируй связь чата с ролью подтверждённым способом, не по заявлению модели. Если надёжной привязки нет, назови ограничение и предложи отдельное решение.

Результат: протокол опыта и инструкция подключения с реальными границами. Приёмка: либо доказано разграничение выбранным механизмом, либо точно описана причина, по которой требуемую гарантию в этом варианте дать нельзя».

10. Реализовать обсуждение и возврат на доработку

Поручение Codex:

«Цель: построить повторяемый обмен замечаниями между ролями. Сообщение содержит адресата, предмет, версию, действие и состояние рассмотрения. Возражение возвращается владельцу материала, пересмотр общего решения — координатору или Егору.

Рамки: чужой файл не переписывай. Сохраняй версии и замечания; при изменении исходника отмечай зависимые результаты для пересмотра. Завершение — принятие, остановка или разрешение спора, без произвольного числа кругов.

Результат: цикл “сценарий → замечание режиссёра → исправление → повторная оценка”. Приёмка: видно, что изменилось, кто принял решение, какие замечания закрыты и что осталось спорным; устаревшая версия не подменяет последнюю принятую».

11. Проверить остановку, сбои и восстановление

Поручение Codex:

«Цель: подтвердить надёжность перед производственным пилотом. Проверь обрыв связи, падение процесса, перезапуск, повтор запроса, ошибку инструмента и результат после отмены. Проверь сохранение и восстановление состояния и файлов.

Рамки: не обещай однократность внешнего действия без гарантии сервиса. Не повторяй действие с неизвестным результатом и риском дубля или расхода. Отмечай частичные результаты; отменённое не возвращай в очередь.

Результат: журнал испытаний, исправления найденных ошибок и короткая инструкция восстановления. Приёмка: после сбоя понятны состояние работы, возможный расход и требуемое решение; восстановление испытано».

12. Выполнить согласованный производственный пилот

Поручение Codex:

«Цель: пройти небольшой полный цикл на согласованном материале. Зафиксируй содержание, формат, критерии качества и инструменты. Платные генерации — только в разрешённых пределах. Не назначай длительность, бюджет или срок.

Рамки: пройди нужные роли, разбор, доработку и сборку. Свяжи сцену с заданием, сценарием, визуальными решениями и замечаниями. Публикация и новые платные сервисы требуют отдельного поручения.

Результат: проверяемый пилот в проекте и краткий отчёт о работе системы. Приёмка: Егор видит результат, основания решений, автоматизированные этапы, нужное участие человека и проблемы перед следующим выпуском».

24. Основания, источники и границы достоверности

24.1. Материалы нашего проекта

Основа восстановлена по `README.md`, «Истории обсуждения», «Разбору Нейта — архитектура ИИ-группы» и дневнику 20 октября 2026 года. Существенные точки: описание киногруппы в 15:13–15:23, требование постоянного пересмотра в 17:56, принятие основы в 18:27, вопросы о ChatGPT и программном диспетчере в 18:48–18:51, уточнение о конкретном чате в 20:54.

Рисунок 20 октября выполнен Егором по предложению помощника. Содержание схемы восстановлено из сохранённого разбора; неясные подписи и закрытая часть листа не использованы для добавления новых обязательных элементов.

Из разбора Нейта используются подтверждённые там различия между встроенной делегацией Codex, советом Roast Room и общей средой Kylon. При подготовке этого руководства ролики заново не проверялись. Собственная схема с диспетчером не приписывается Нейту, а продукты Roast Room и Kylon не объявляются выбранными для установки.

24.2. Прочитанная официальная документация

Страницы проверены 25 октября 2026 года. Ссылки ведут на источники конкретных механизмов; доступность для нашей учётной записи не проверялась подключением производственного контура.

Источник	Для чего использован
Проекты и чаты	Общий контекст проекта и различие истории разговоров и локальных файлов
Справочник MCP-интеграций	`openai/session`, другие метаданные, описания инструментов
Авторизация MCP	Раздельная проверка пользователя, токенов и полномочий
Подключение и проверка	Реальное подключение MCP к ChatGPT и испытание инструментов
Secure MCP Tunnel	Частный сервер, исходящий канал, требования доступа и предел запроса
Codex SDK	Программное управление Codex через Python
Codex app-server	Сессии, ходы, события, возобновление и прерывание
Неинтерактивный Codex	Запуск задания командой и продолжение конкретной истории
Разрешения Codex	Различие рабочей папки, ограничений чтения, записи и MCP
MCP в Codex	Подключение инструментов к исполнителям
Авторизация Codex	Способы входа и граница между подпиской и API

24.3. Что является проектным предложением

Структура папок, схема таблиц, названия инструментов, карточки заданий, модель очереди, регистрация разговоров, процессы приёмки и порядок этапов разработаны для «Мульт Япония» в этом руководстве. Это не описание готового продукта OpenAI и не утверждение, что такие инструменты уже доступны после одного подключения.

24.4. Что предстоит доказать в нашей среде

- Работа канала ChatGPT → наш MCP-сервер в нужной учётной записи.
- Пригодность метаданных для устойчивой привязки конкретных разговоров и доверие к их происхождению.
- Реальное разделение чтения и записи между ролями всеми доступными им способами.
- Запуск, продолжение и остановка исполнителей с сохранением правильных сессий.
- Согласованность очереди, файлов, версий и журнала после сбоев.
- Качество производственного пилота и фактическая стоимость выбранных инструментов.